Counting Challenge

Object counting with instance masks: a classical (Non-AI) and a learned (AI) solution

Om Rode

November 2026

Abstract

The task is to count the screws, bolts, discs and nuts in 25 photographs and overlay one mask per object, with an accuracy above 95 %. Two independent solutions were built. The **Non-AI** solution uses classical OpenCV operations: illumination flattening, Otsu thresholding, round-hole detection for nuts, a disc model based on the distance transform, and area-based cluster counting with watershed splitting. The **AI** solution is a YOLOv8s-seg instance-segmentation model, fine-tuned on labels generated automatically with SAM 2.1. An automatic audit pointed a human only at suspicious spots; 16 objects in 8 images were corrected with one click each. Count accuracy is $1 - |\hat{n} - n|/n$ per image. The Non-AI solution reaches **99.7 %** mean accuracy over all images (worst image 98.0 %). The AI solution reaches **99.9 %** on the five held-out test images and 99.4 % over all images (worst image 95.2 %).

1 Dataset and ground truth

The dataset has 25 photographs of metal parts on a plain background, in two groups (Figure 1):

- `Screws_2024_07_15`: seven 1920 × 1080 shots of the same 43 screws. The file name `img1_43_nosy` gives the count.
- `ScrewAndBolt_20240713`: eighteen 3000 × 4000 phone photos. They show a few screws (10 and 21), dense piles of countersunk screws (about 193), screws mixed with discs (about 98), and a pile of 354 hex nuts.

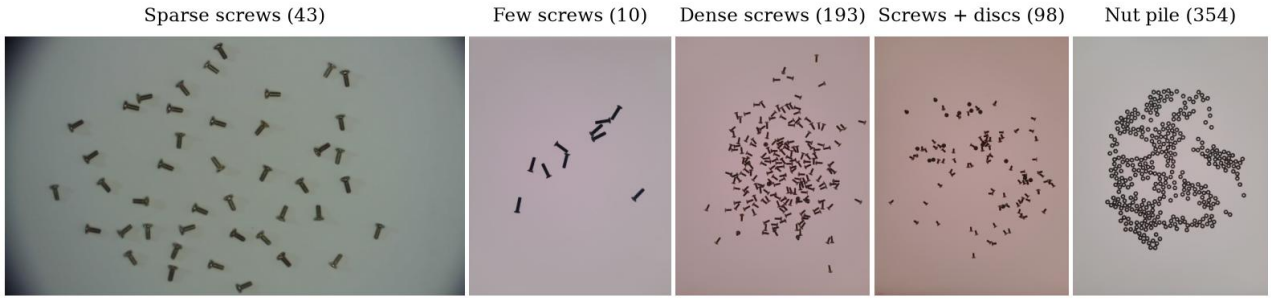


Figure 1: One image of each scene type. These five images form the held-out test split of the AI model.

Ground truth. The dataset has no labels, so the ground truth (`Om_Rode/ground_truth.csv`) was built from these sources, which agree with each other:

- manual counts of the sparse images, and the “43” in the file name;
- the number of round nut holes, which is the same in all five nut photos (354);
- the SAM auto-labels, checked by the audit and corrected where needed (Section 3.1), then approved on a review page.

2 Non-AI solution (OpenCV)

`Non_AI/count_objects.py` has no learned parameters. It processes one image in about 0.4 s on a CPU.

2.1 Foreground segmentation

1. The image is resized to a long side of 1600 px, so that all kernel sizes are the same for every photo.
2. **Illumination flattening.** The background is estimated with a grey-level closing (41×41 ellipse), which removes the dark objects, followed by a large Gaussian blur. The image is divided by this estimate. This removes uneven lighting and lens vignetting (Figure 2b).
3. **Otsu threshold** on the flattened image, a 3×3 opening and closing, and hole filling (Figure 2c).
4. **Valid field of view.** Blobs that are more than 30 % inside the black vignette corners are removed.

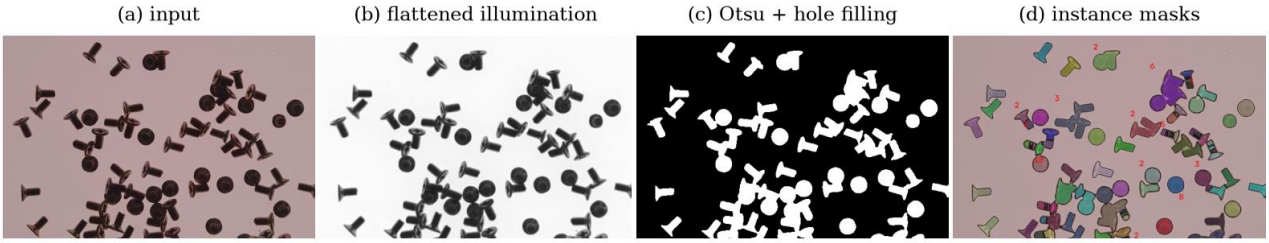


Figure 2: Non-AI pipeline on a screws-and-discs image (crop). (d) shows one coloured mask per counted object; a red number is the count given to a cluster.

2.2 Counting

Nuts: one round hole is one nut. The inner contours of the thresholded image are the holes. A hole counts if it is round (circularity $4\pi A/P^2 > 0.75$) and between 0.4 and 2 times the median hole area. Gaps that are enclosed between touching parts are irregular, so they fail the circularity test. A blob with k round holes contains k nuts. Each pixel of the blob is assigned to its nearest hole centre, which splits the blob into k masks (a Voronoi split, Figure 3). On the five nut photos this gives 353–355 against a ground truth of 354.

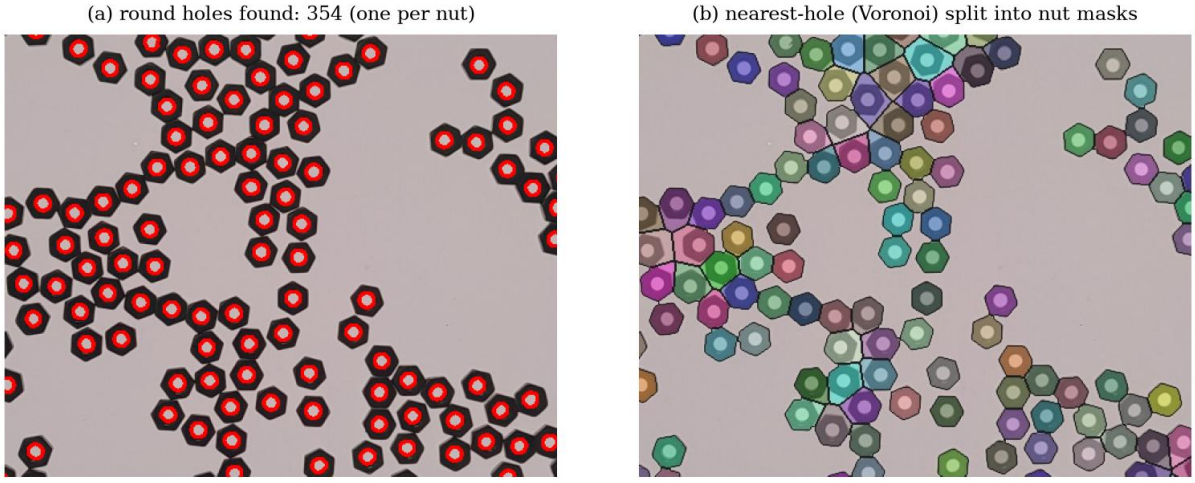


Figure 3: Nut pile (crop). (a) Detected round holes. (b) Nut masks after the nearest-hole split.

Screws and discs.

- The *single-object area* a_1 is the left-most strong mode of the log-area histogram of all blobs. The median does not work, because in sparse scenes most blobs are touching pairs.
- *Discs* have about 1.3 times the area of a screw. Isolated round blobs (circularity > 0.8) give the disc area a_d and radius r. Inside any blob, each region where the distance transform is at least $0.8\,r$ is one disc (Figure 4). Screw heads and shanks are much thinner, so they do not produce such regions.

- The count of a blob is

$$k = n_d + \operatorname{round}\left(\frac{A - n_d\, a_d}{a_1 \cdot c}\right), \qquad c = \begin{cases} 1.06 & \text{if the blob holds more than 2.5 objects} \\ 1 & \text{otherwise} \end{cases}$$

  where A is the blob area and n_d the number of discs. The crowding factor c was measured: in clusters of three or more objects, the area per object is 1.05–1.07 times a_1, because shadows and blurred edges between touching objects add area.
- A cluster is split into k masks with a distance-transform watershed.

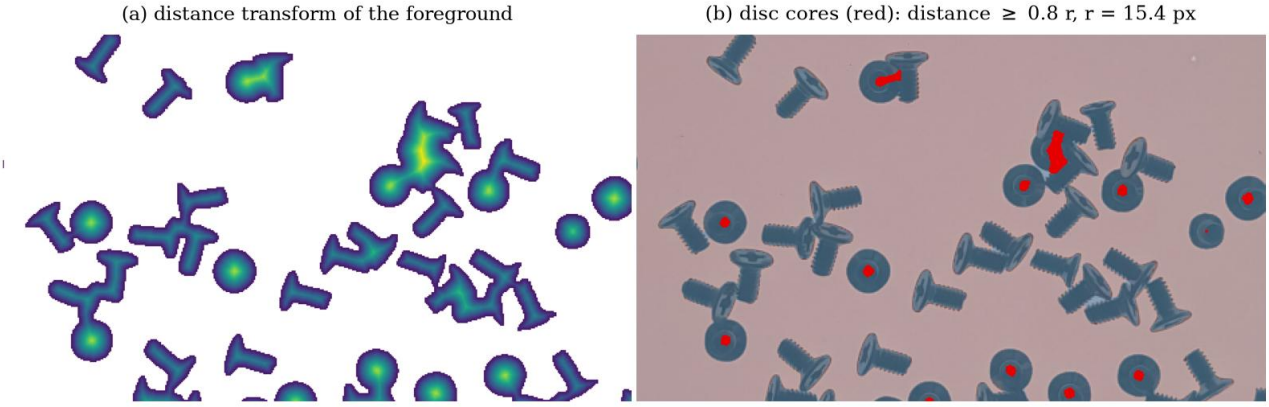


Figure 4: Disc detection inside clusters. Only discs have a distance-transform core thicker than $0.8\,r$ (red). Two touching discs give one elongated core, which is counted as two.

Before the disc model and the crowding factor were added, every disc was counted as about 1.3 objects. The screws-and-discs images then scored only 84–94 %. With both changes, the worst image is at 98 %.

3 AI solution (YOLOv8s-seg trained on SAM 2.1 labels)

3.1 Automatic labelling with SAM 2.1

`AI/autolabel_sam.py` produces instance labels without manual drawing:

1. The image is resized to a long side of 2400 px. SAM 2.1-base runs in "segment everything" mode on 1024×1024 tiles with stride 768. Masks that touch an interior tile edge are dropped, because the neighbouring tile sees those objects whole.
2. **Clean-up:**
   - duplicate masks are removed by IoU;
   - a mask is kept only if it lies on the Non-AI foreground and inside the valid field of view;
   - group masks and part masks are resolved;
   - fragments smaller than $0.2\times$ the median object area are removed.
3. **Recovery of missed objects (up to 3 rounds).** Each uncovered thick dark region is prompted with a point. If that fails, it is prompted with a point and a box. If the region is still too large, it gets one prompt per distance-transform peak, with the other peaks as negative points.
4. **Merging parts.** SAM often returns a pan-head screw as two masks (head and shank). Adjacent parts whose combined area is close to one object are merged.

The labels are stored as YOLO polygons, as compressed masks for the audit, and as overlays (Figure 5).

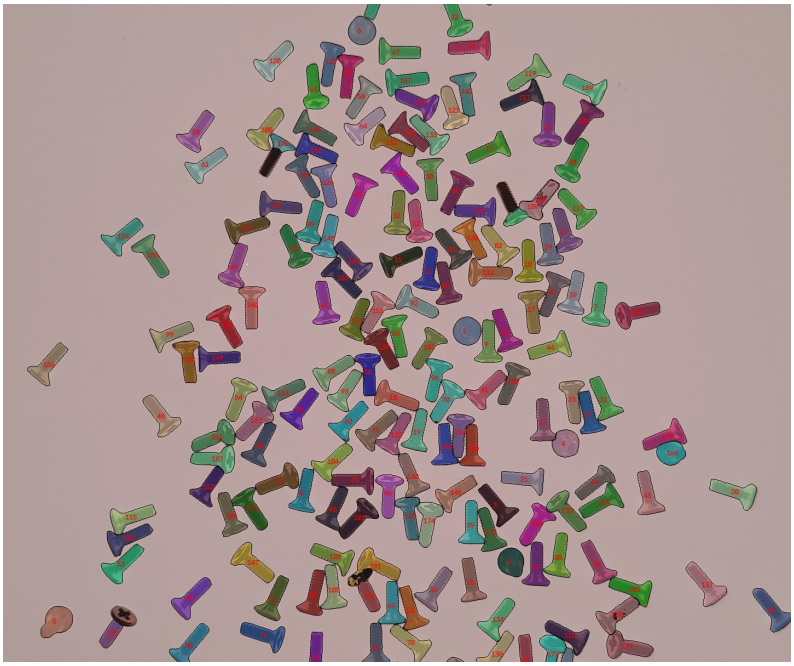


Figure 5: Final SAM labels for a dense screw image (crop). Each colour is one instance.

3.2 Audit and minimal human correction

`AI/audit_labels.py` compares the labels with independent classical signals and flags these cases:

- *possible miss*: a thick foreground blob with no mask;
- *possible merge*: a mask larger than 1.6× the median area;
- a mismatch with the number of nut holes or with the Non-AI count.

Only the flagged spots were reviewed. 16 of them were real errors: 12 merges and 4 misses. The others were harmless, for example a screw head masked without its shank, which does not change the count. Each error was fixed with one click in `corrections.json`, applied by `apply_corrections.py`:

- `add`: a SAM point prompt at the click;
- `split`: one prompt per distance-transform peak of the merged mask, with the other peaks as negative points.

Table 1 lists the corrections and Figure 6 shows two of them. 17 of the 25 images needed no correction. Before correction, the SAM count was 0.9 % from the final count on average, over the 18 phone photos.

Table 1: One-click label corrections. All other images were correct automatically.

Code	Image	Adds	Splits	SAM automatic	After review
B09	20240713_193135	1	0	20	21
B12	20240713_193831	1	2	190	193
B13	20240713_193839	0	1	194	195
B14	20240713_193907	0	5	188	193
B16	20240713_194206	2	0	97	99
B18	20240713_194232	0	1	97	98
B19	20240713_194256	0	2	96	98
B20	20240713_194316	0	1	98	99
Total (8 images)		4	12		

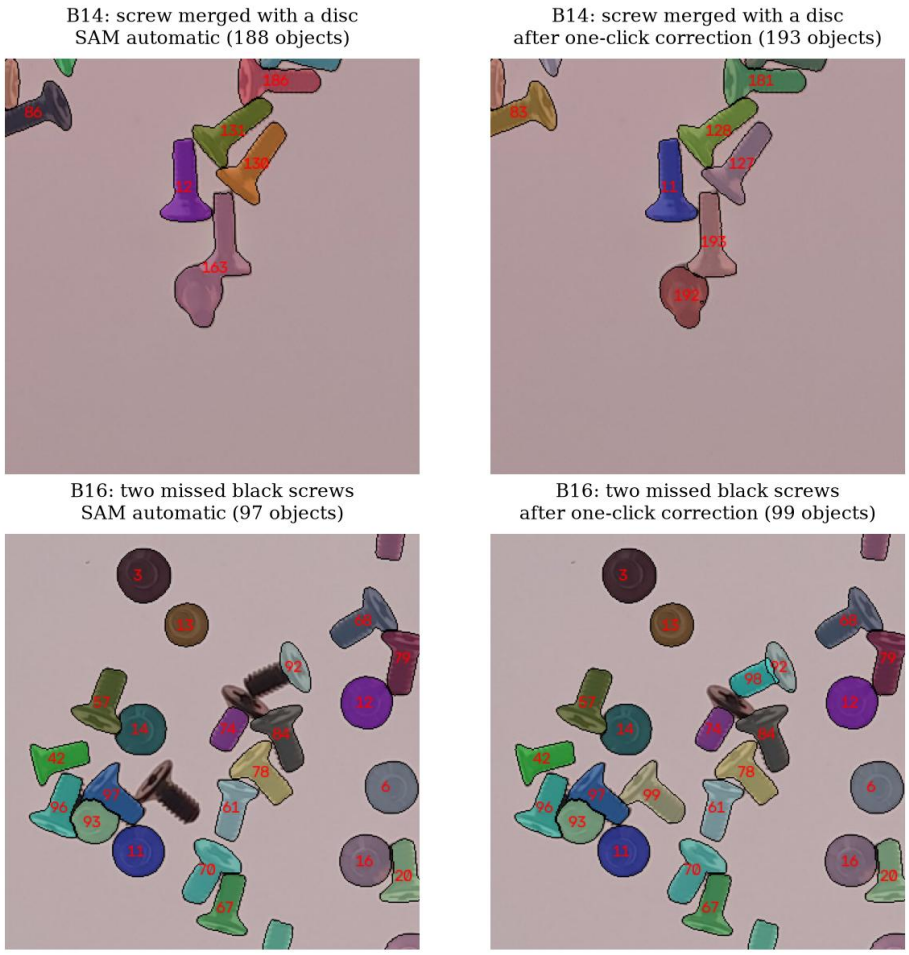


Figure 6: Two corrections. Top: a screw merged with a disc is split with two prompts. Bottom: two black screws that SAM missed are added with one point each.

3.3 Dataset and training

Split by image, so that no tiles of a test image are used in training:

- test: one image per scene type (Figure 1);
- validation: `img6` and `194206`;
- train: the remaining 18 images.

The training images are cut into 1024 × 1024 tiles with stride 512, at the 2400 px scale used for inference. This gives 182 tiles. An object cut by a tile border is kept if at least 40 % of it is visible. Table 2 lists the training settings and Figure 7 the training curves.

Table 2: Training configuration (`AI/train.py`).

Model	YOLOv8s-seg, COCO-pretrained, 11.8 M parameters, 1 class (*object*)
Input	1024 × 1024 tiles, batch 1
Epochs	at most 120, early stopping with patience 40; stopped at epoch 86, best weights from epoch 46
Augmentation	rotation ±180°, horizontal and vertical flips, scale ±40 %, translation 0.1, HSV jitter; mosaic off
Other	`max_det=1000`, `overlap_mask=True`, seed 0
Hardware	NVIDIA RTX 3050 Laptop (6 GB); 29 min of training

Why mosaic is off and the batch is 1. A nut tile holds about 300 objects. Mosaic joins four tiles into one image, which gives about 1,200 objects. The per-instance mask loss for that many objects needs more

than 6 GB of GPU memory; training ran out of memory at batch 4 and at batch 2. The random tile crops and rotations already give enough variation.

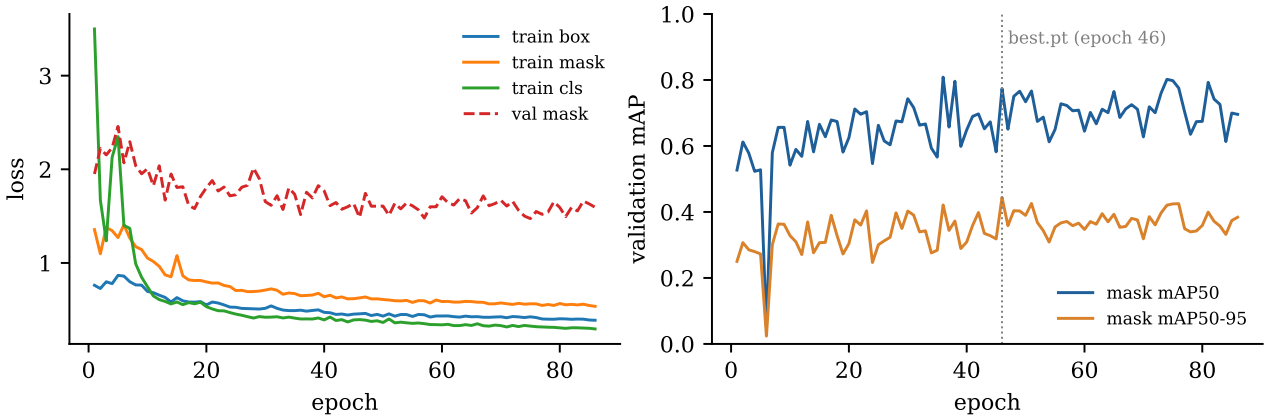


Figure 7: Training losses and validation mask mAP.

The validation mAP (mask mAP50 = 0.77) is lower than the count accuracy. The reason is that YOLO validates on whole images downscaled to 1024 px, where the objects are about 2.3 times smaller than at inference.

3.4 Inference

`AI/count_objects_yolo.py` resizes the image to a long side of 2400 px and predicts on the whole image in one pass. The input size is rounded up to a multiple of 32. The settings are `conf=0.4`, `iou=0.5`, `max_det=1000` and full-resolution masks. The count is the number of predicted masks.

4 Results

Table 3: Mean and worst-image count accuracy (%) for each split. The AI model never saw the val and test images during training. The Non-AI solution has no training, so for it all splits are unseen data.

		Non-AI		AI	
Split	Images	Mean	Worst	Mean	Worst
train	18	99.66	98.0	99.29	95.2
val	2	100.00	100.0	99.49	99.0
test	5	99.79	99.0	99.90	99.5
all	25	99.71	98.0	99.43	95.2

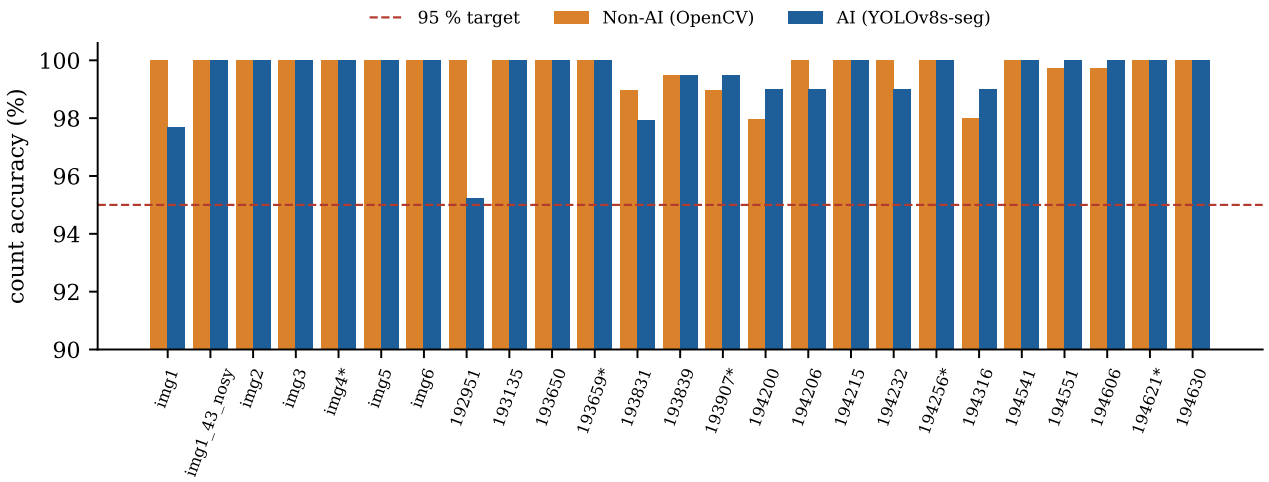


Figure 8: Count accuracy for each image. Test images are marked with *. Every image is above the 95 % target with both methods.

Table 4: Counts for each image. “SAM” is the automatic label count before human correction.

Image	Split	Ground truth	SAM	Non-AI	Acc. (%)	AI	Acc. (%)
img1	train	43	43	43	100.0	44	97.7
img1_43_nosy	train	43	43	43	100.0	43	100.0
img2	train	43	43	43	100.0	43	100.0
img3	train	43	43	43	100.0	43	100.0
img4	test	43	43	43	100.0	43	100.0
img5	train	43	43	43	100.0	43	100.0
img6	val	43	43	43	100.0	43	100.0
20240713_192951	train	21	21	21	100.0	22	95.2
20240713_193135	train	21	20	21	100.0	21	100.0
20240713_193650	train	10	10	10	100.0	10	100.0
20240713_193659	test	10	10	10	100.0	10	100.0
20240713_193831	train	193	190	195	99.0	197	97.9
20240713_193839	train	195	194	196	99.5	194	99.5
20240713_193907	test	193	188	195	99.0	194	99.5
20240713_194200	train	98	98	100	98.0	99	99.0
20240713_194206	val	99	97	99	100.0	98	99.0
20240713_194215	train	98	98	98	100.0	98	100.0
20240713_194232	train	98	97	98	100.0	99	99.0
20240713_194256	test	98	96	98	100.0	98	100.0
20240713_194316	train	99	98	97	98.0	98	99.0
20240713_194541	train	354	354	354	100.0	354	100.0
20240713_194551	train	354	354	355	99.7	354	100.0
20240713_194606	train	354	354	353	99.7	354	100.0
20240713_194621	test	354	354	354	100.0	354	100.0
20240713_194630	train	354	354	354	100.0	354	100.0

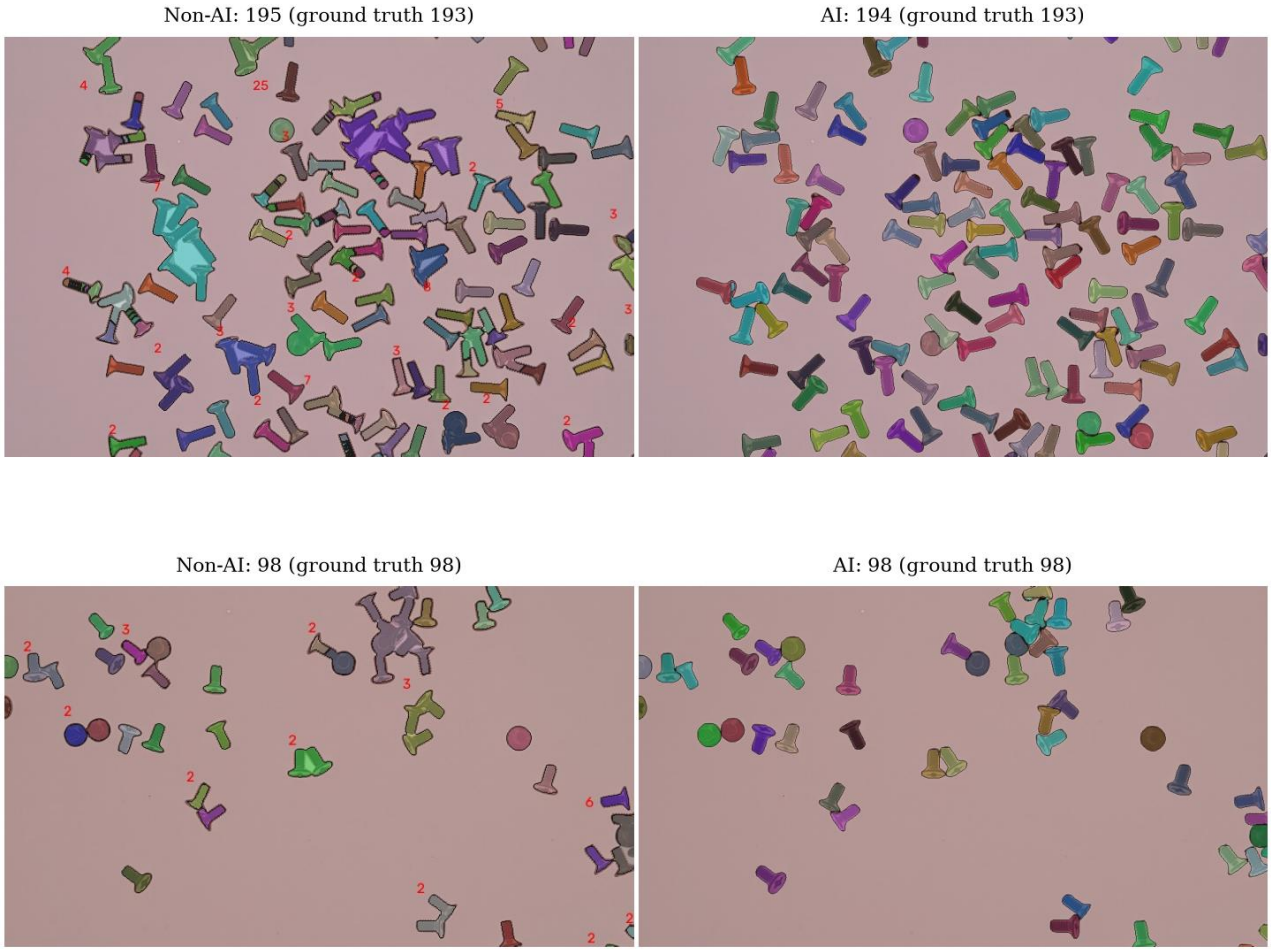


Figure 9: Outputs on two test images (crops): dense screws (top), and screws with discs (bottom). Left: Non-AI. Right: AI.

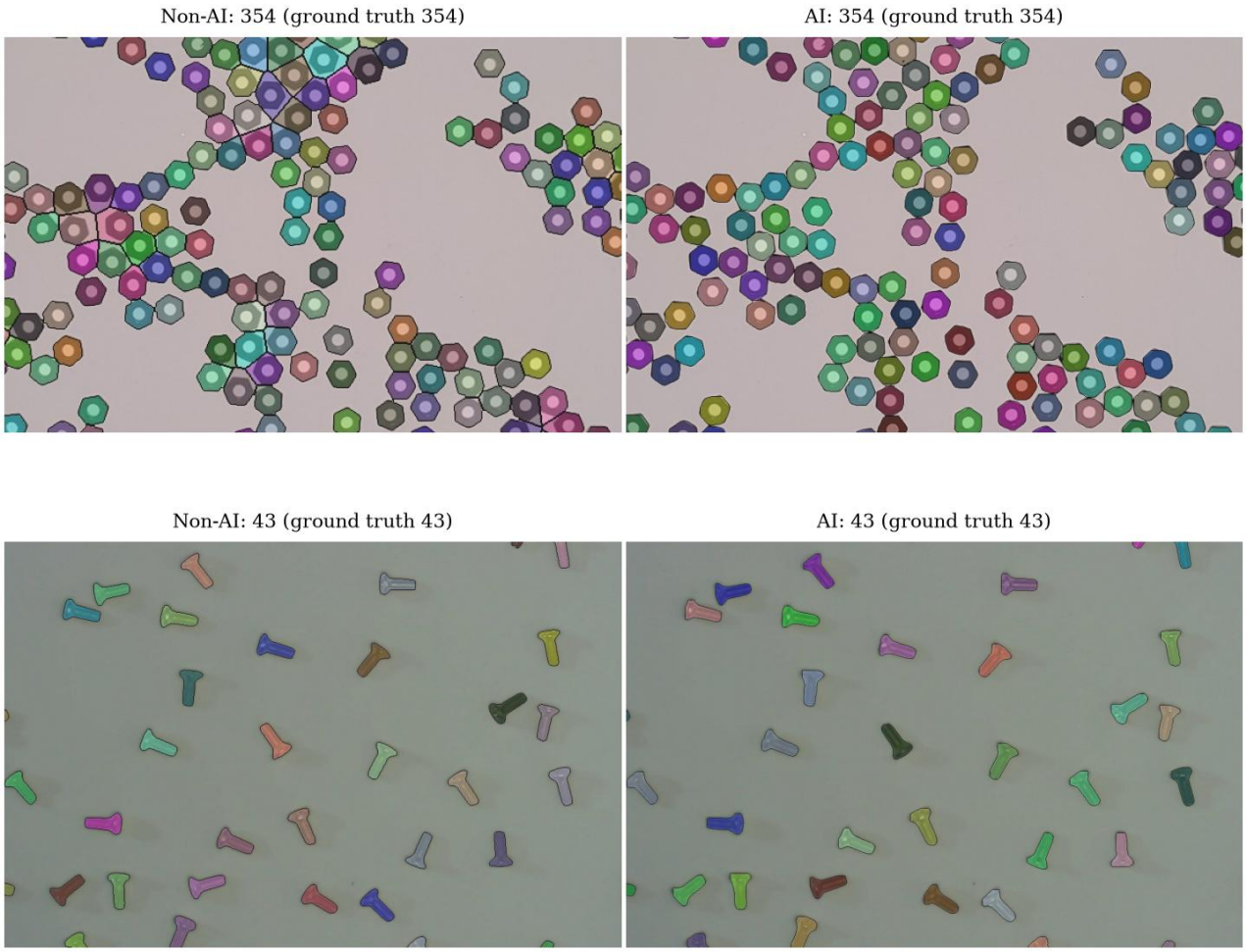


Figure 10: Outputs on two more test images (crops): nut pile (top) and sparse screws (bottom).

5 Discussion

- **Counts.** Both methods are above 95 % on every image. Each has its own failure modes. The Non-AI method depends on the area of one object, so it would fail on a scene with no isolated objects. The largest AI error is one extra detection on a sparse training image (22 against 21).
- **Mask quality.** The AI masks follow the object outlines, even inside dense piles (Figure 9). The Non-AI watershed counts clusters correctly, but its boundaries inside a cluster are only approximate.
- **Labelling effort.** SAM with the audit made the AI solution possible with very little manual work: 16 clicks for about 3,300 objects.
- **Limitations.** The ground truth comes from the same SAM labels that the AI model is trained on, so a systematic SAM error would not show up in the AI test score. The independent checks reduce this risk: the Non-AI counts, the nut-hole counts, and manual review of every flagged spot.

6 Reproducing

```
pip install -r Om_Rode/Non_AI/requirements.txt -r Om_Rode/AI/requirements.txt
python Om_Rode/Non_AI/count_objects.py --data data            # Non-AI
python Om_Rode/AI/autolabel_sam.py && python Om_Rode/AI/audit_labels.py
python Om_Rode/AI/apply_corrections.py                         # 16 clicks
python Om_Rode/AI/prepare_dataset.py && python Om_Rode/AI/train.py
python Om_Rode/AI/count_objects_yolo.py --data data            # AI
python Om_Rode/evaluate.py Om_Rode/AI/outputs/counts.csv
python Om_Rode/report/make_figures.py                          # this report
```